

報告文件

Periodic task set 產生方式說明

task_generator.py 整體執行流程：

1. 初始化與環境設定

- 程式一開始設定了排程總時間 $H = 72$ 。
- 可設定固定隨機種子 SEED，確保每次執行程式產生的任務集都是一模一樣的（開發過程方便除錯與重現結果用）。
- 執行 `project_root()` 自動定位專案根目錄，確保後續產出的檔案能正確存入 `output/` 資料夾。

2. 主程式啟動與生成迴圈

- 程式進入主區塊 `if __name__ == "__main__":` 後，會呼叫 `generate_periodic_tasks()` 。
- 這個函式內部是一個 `while True` 的無窮迴圈。由於隨機生成的參數很難一次就完美符合所有複雜的數學限制，程式會不斷嘗試生成新任務集，直到有一組成功通過所有驗證為止。

3. 嘗試生成任務 (`_try_to_generate_period_tasks`)

在這個階段，程式會進行初步的隨機生成，並強制介入以滿足部分硬性規定：

- **決定數量：** 隨機決定產生 6 到 10 個任務。
- **給定初始值：** 隨機給予 `release time ( r )`、`period ( p )`、`execution time ( e )`、`deadline ( d )`、`energy ( w )` 與是否可搶佔 (`preempt`) 。
- **強制滿足規定 1-4 (執行時間)：** 程式利用索引，強制前兩個任務的 $e=2$ ，第三個任務的 $e=3$ 或 4 ，以滿足「至少兩個 $e=2$ ，至少一個 $e \geq 3$ 」的規則。
- **強制滿足規定 1-4 (高耗能)：** 檢查 $w \geq 14$ 的任務是否足夠，若不夠，直接挑選任務將其 w 強制改為 14 到 18 之間。
- **強制滿足規定 1-6 (緊湊時限)：** 計算出 20% 的數量，強制將這些任務的 `deadline ( d )` 設為與 `execution time ( e )` 相等。
- **強制滿足規定 1-7 (不可搶佔)：** 找出執行時間 $e > 1$ 的任務，隨機挑選兩個強制設為不可搶佔 (`preempt = 0`) 。

4. 內部核心驗證 (`validate_tasks`)

初步生成的任務集會送入這裡，檢查那些「無法靠單一任務強制設定，必須看整體數據」的複雜規則：

- **檢查總工作量 (1-3)：** 模擬 72 小時內這些任務會釋放幾次，加總確認總工作數量 (Jobs) 是否大於 30。

- **檢查多樣性 (1-4)：** 確認所有任務中至少包含 3 種不同的 Period 數值。

- **檢查工作負載密度 (1-5)：** 計算

$\sum (e / p)$ ，確認密度落在 0.7 到 1.0 之間。

- **尋找合法的 Frame Size (1-8)：** 這是最困難的一步。程式會遍歷 72 的所有因數 (1, 2, 3, 4, 6...)，檢查是否有任何一個數字 f 同時滿足：

- 大於等於最大的 e
- 小於等於最小的 p
- 對所有任務都滿足排程理論公式：

$$2f - \gcd(f, p) \leq d$$

只要上述任一條件失敗，這組任務就會被丟棄，回到步驟 3 重新生成。若全數通過，則將找到的 f 記錄為 `selected_f` 並回傳成功。

5. 輸出與最終雙重確認

- 當迴圈終於找到一組合法的任務集後，程式會將其格式化並印在終端機上。
- 將結果寫入到 `output/task_set.json` 檔案中。
- 最後，呼叫 `final_validate_tasks()` 進行最後一次的驗證，他會輸出詳細的錯誤訊息以做到防呆機制。

放寬 Assumption 的限制式建模 [Only Level 2]

放寬 Assumptions 的限制式建模

放寬的 assumptions 包含：

1. 再生能源出力不確定性
2. 儲能設備真實運作情境
3. 彈性市場機制中的 day-ahead sell commitment 與 cancellation penalty

Level 2 新增的輔助參數

符號	說明
$A_{r,t}$	renewable r 在時間 t 的 actual available ratio
$\theta_{r,t}$	renewable forecast uncertainty factor
$Chg_{b,t}$	battery b 在時間 t 的充電量，由 $k_{j,i,t}$ 推導
$Dis_{b,t}$	battery b 在時間 t 的放電量，由 $P_{b,t}$ 表示
$SOC'_{b,t}$	battery b 自放電後、充放電前的 SOC
$SOC''_{b,t}$	battery b 放電後、充電前的 SOC
S_t^{DA}	時間 t 的 day-ahead sell commitment
$Short_t$	時間 t 的售電承諾不足量
C^{age}	battery aging cost
C^{cancel}	sell commitment cancellation penalty

放寬 Assumption 1：再生能源出力不確定性

Level 1 假設再生能源出力完全依照日前預測值，不考慮預測誤差。因此，再生能源

r 在時間 t 的最大可用電量由 forecast ratio $F_{r,t}$ 決定：

$$0 \leq P_{r,t} \leq Cap_r F_{r,t}$$

其中：

符號	說明
Cap_r	renewable r 的額定容量
$F_{r,t}$	renewable r 在時間 t 的 forecast ratio

Level 2 放寬方式

Level 2 中，實際 renewable output 不再直接等於 forecast，而是引入 uncertainty factor

$\theta_{r,t}$ 。因此，renewable r 在時間 t 的 actual available ratio 定義為：

$$A_{r,t} = \min (1, \max (0, F_{r,t} \theta_{r,t}))$$

其中：

符號	說明
$\theta_{r,t}$	renewable r 在時間 t 的 forecast uncertainty factor
$A_{r,t}$	renewable r 在時間 t 的 actual available ratio

實際可用 renewable energy 為：

$$Available^{actual} * r, t = Cap_r A * r, t$$

因此，Level 2 的 renewable output constraint 改為：

$$0 \leq P_{r,t} \leq Cap_r A_{r,t}$$

在 Level 2 中，再生能源的實際可用量不再直接使用日前預測值，而是根據 forecast ratio 與 uncertainty factor 計算 actual available ratio。若實際出力低於預測值，系統不能假設 renewable 仍有 forecast 中的可用電量，若實際出力高於預測值，系統可以使用更多 renewable energy 供應 jobs、battery charging 或售電。

演算法實作對應

在 advanced_scheduler.py 中，build_actual_renewable() 會根據 forecast ratio 與 uncertainty_factor() 計算 actual_ratio 與 actual_available_mwh。在 dispatch_advanced_energy() 中，排程器會使用 actual renewable availability 進行 dispatch，而不是直接使用 forecast renewable output。

Forecast uncertainty factor 的設定

透過 forecast uncertainty factor

$\theta_{r,t}$ ，使 actual renewable output 不再固定等於 forecast output。

$$\theta_{r,t} = 1 + 0.1 \sin\left(\frac{2\pi t}{24}\right)$$

由於：

$$-1 \leq \sin\left(\frac{2\pi t}{24}\right) \leq 1$$

因此：

$$0.9 \leq \theta_{r,t} \leq 1.1$$

也就是 actual renewable output 會在 forecast 的

±10% 範圍內每小時變動。

actual renewable ratio 定義為：

$$A_{r,t} = \min(1, \max(0, F_{r,t}\theta_{r,t}))$$

因此 Level 2 的 renewable output constraint 為：

$$0 \leq P_{r,t} \leq Cap_r A_{r,t}$$

透過

$\theta_{r,t}$ 讓 $A_{r,t}$ 不一定等於 $F_{r,t}$ ，因此放寬了 forecast 完全準確的 assumption。

放寬 Assumption 2：儲能設備真實運作情境

Level 1 中，儲能設備的模型較理想化。核心排程主要使用 thermal generators 作為供電來源，battery 在基本 energy balance 中並未完整考慮效率損失、老化成本、自放電、cycle limit 或 SOC-dependent power limit。

若以簡化 battery model 表示，Level 1 可近似為：

$$SOC_{b,t} = SOC_{b,t-1} + Chg_{b,t} - Dis_{b,t}$$

此式代表 battery 充放電皆無損耗，也不產生老化成本。

Level 2 中，本設計放寬此 assumption，加入以下 8 項 battery operation constraints：

1. charging efficiency
2. discharging efficiency
3. self-discharge
4. battery throughput limit
5. battery aging cost
6. SOC-dependent charge limit
7. SOC-dependent discharge limit
8. no simultaneous charging and discharging

Battery charging amount 與 discharging amount

為了描述 battery 的充電與放電行為，先定義兩個輔助量，其中

$Chg_{b,t}$ 表示 battery b 在時間 t 接收到的總充電量， $Dis_{b,t}$ 表示 battery b 在時間 t 對外釋放的總放電量。

Battery charging amount 由 charging jobs 的

$k_{j,i,t}$ 推導而來：

$$Chg_{b,t} = \sum_{\substack{j \in J_{chg} \\ b(j) = b}} \sum_{i \in I_g \cup I_r} k_{j,i,t}$$

其中，

$b(j) = b$ 表示 charging job j 對應到 battery b 。

Battery discharging amount 由 battery output

$P_{b,t}$ 表示：

$$Dis_{b,t} = P_{b,t}$$

1. Charging efficiency

Level 1 中，充電可視為無損耗：

$$SOC_{b,t} = SOC_{b,t-1} + Chg_{b,t} - Dis_{b,t}$$

也就是充入

$Chg_{b,t}$ 的電量，會完整增加到 battery 的 SOC。

Level 2 放寬方式

Level 2 中加入 charging efficiency。本設計設定：

$$\eta^{ch} = 0.95$$

也就是 battery 的充電效率為 95%。當 battery 充電時，只有

95% 的輸入電量會實際存入 SOC，剩下 5% 視為充電過程中的能量損失。

因此 Level 2 的 SOC 更新式為：

$$SOC_{b,t} = SOC_{b,t}^{sd} + 0.95Chg_{b,t}$$

其中，

$SOC_{b,t}^{sd}$ 表示 self-discharge 後的 SOC， η^{dis} 表示放電效率。

當 battery 充電時，輸入到 charging job 的電量不會 100% 存入 battery。本設計設定 charging efficiency 為

0.95，代表每充入 1 MWh，實際只有：

$$1 \times 0.95 = 0.95 \text{ MWh}$$

會增加到 SOC；剩下：

$$1 - 0.95 = 0.05 \text{ MWh}$$

視為充電損耗。

2. Discharging efficiency

Level 1 中，放電可視為無損耗：

$$SOC_{b,t} = SOC_{b,t-1} + Chg_{b,t} - Dis_{b,t}$$

也就是 battery 對外輸出

$Dis_{b,t}$ 的電量時，SOC 只會減少 $Dis_{b,t}$ 。

Level 2 放寬方式

Level 2 中加入 discharging efficiency。本設計設定：

$$\eta^{dis} = 0.90$$

也就是 battery 的放電效率為 90%。當 battery 放電時，若要對外輸出

$Dis_{b,t}$ ，battery 內部實際需要消耗更多電量，SOC 實際扣除量為：

$$\frac{Dis_{b,t}}{0.90}$$

因此 Level 2 的 SOC 更新式為：

$$SOC_{b,t} = SOC^{sd} * b, t + 0.95Chg * b, t - \frac{Dis_{b,t}}{0.90}$$

其中，

$SOC_{b,t}^{sd}$ 表示 self-discharge 後的 SOC， η^{ch} 表示充電效率。

當 battery 放電時，輸出的電量不會等於 SOC 實際減少的電量。本設計設定 discharging efficiency 為

0.90，代表若 battery 對外輸出 1 MWh，SOC 實際需要扣除：

$$\frac{1}{0.90} = 1.111 \text{ MWh}$$

也就是約有

0.111 MWh 在放電過程中損失，因此 battery 放電不再是理想無損耗過程。

3. Self-discharge

Level 1 不考慮 self-discharge，因此 battery 在沒有充放電時，SOC 不會自然下降：

$$SOC_{b,t}^{sd} = SOC_{b,t-1}$$

Level 2 放寬方式

Level 2 中，battery 在每個時間點會先發生 self-discharge，其中

ρ_b 是 battery b 的 self-discharge rate。

$$\rho_b = 0.0005$$

也就是 battery 每小時 self-discharge rate 為

0.05%。因此 self-discharge 後的 SOC 為：

$$SOC_{b,t}^{sd} = \max(SOC_b^{min}, (1 - 0.0005)SOC_{b,t-1})$$

其中，

$SOC_{b,t}^{sd}$ 表示 self-discharge 後、充放電前的 SOC。

即使 battery 沒有被使用，其 SOC 也會隨時間自然衰減，為避免 SOC 低於最低容量，self-discharge 後的 SOC 不得小於

SOC_b^{min} 。

4. Battery throughput limit

Level 1 不限制 battery 在整個排程期間的總使用量。也就是說，只要每小時的 charge / discharge power 與 SOC bound 滿足，battery 理論上可以被頻繁充放電。

Level 2 放寬方式

Level 2 限制 battery 在整個排程期間的總充放電量，設定

R^{cycle} 是 battery throughput ratio：

$$R^{cycle} = 1.20$$

因為 battery 在排程期間允許使用的總充放電量，最多為其可用容量範圍的

1.20 倍。

定義 battery throughput limit：

$$L_b = (SOC_b^{max} - SOC_b^{min}) \times 1.20$$

因此 battery 不能在排程期間被無限制反覆充放電，每一次 charging 與 discharging 都會消耗 battery throughput budget，因此總使用量不得超過

L_b 。

所以總充放電量必須滿足：

$$\sum_{t \in T} (Chg_{b,t} + Dis_{b,t}) \leq L_b$$

可避免 battery 被過度使用或進行不合理套利。

5. Battery aging cost

Level 1 不考慮 battery aging cost，因此 battery usage 不會增加 objective cost：

$$C^{age} = 0$$

Level 2 放寬方式

Level 2 中，battery 使用會造成 aging cost。本設計設定：

$$c^{age} = 5.0$$

其中，

c^{age} 表示每 MWh battery throughput 的 aging cost。也就是 battery 每充電或放電 1 MWh，會產生 5 元的 aging cost。

因此，Level 2 的 battery aging cost 定義為：

$$C^{age} = 5.0 \sum_{b \in I_b} \sum_{t \in T} (Chg_{b,t} + Dis_{b,t})$$

其中，

$Chg_{b,t} + Dis_{b,t}$ 表示 battery b 在時間 t 的 battery throughput，也就是該時間點的總使用量。

因為battery 充電與放電都會造成老化成本，所以 battery usage 不再是免費資源。此成本會加入 objective，使系統在追求售電收益時，同時考慮 battery degradation，避免為了提高短期售電收益而過度使用 battery。

6. SOC-dependent charge limit

Level 1 中，battery charging 主要只受到固定最大充電量限制：

$$Chg_{b,t} \leq Chg_b^{max}$$

也就是說，只要充電量不超過 battery 每小時最大充電能力，就可以進行充電。

Level 2 放寬方式

Level 2 中，battery 的可充電量會受到目前 SOC 狀態影響。當 battery 越接近滿電時，能再充進去的電量應該越少。

本設計在程式中設定：

$$\gamma^{ch} = 0.60$$

$$\eta^{ch} = 0.95$$

其中，

γ^{ch} 是 SOC-dependent charge ratio， η^{ch} 是 charging efficiency。

先定義放電後、充電前的 SOC：

$$SOC_{b,t}^{pre} = SOC_{b,t}^{sd} - \frac{Dis_{b,t}}{\eta^{dis}}$$

接著，Level 2 的充電限制式為：

$$Chg_{b,t} \leq \min (Chg_b^{max}, 0.60 \cdot \frac{SOC_b^{max} - SOC_{b,t}^{pre}}{0.95})$$

此限制式表示 battery 的充電量同時受到兩個條件限制：

1. 不可以超過固定最大充電量 Chg_b^{max} 。
2. 不可以超過目前 SOC 剩餘容量所允許的充電量。

當

$SOC_{b,t}^{pre}$ 越接近 SOC_b^{max} 時， $SOC_b^{max} - SOC_{b,t}^{pre}$ 會越小，因此可充電量也會下降。進一步考慮目前 SOC 狀態，因此 battery 越接近滿電時，可充電量越小。

7. SOC-dependent discharge limit

Level 1 中，battery discharge 主要只受到固定最大放電量限制：

$$Dis_{b,t} \leq Dis_b^{max}$$

也就是說，只要放電量不超過 battery 每小時最大放電能力，就可以進行放電。

Level 2 放寬方式

Level 2 中，battery 的可放電量會受到目前 SOC 狀態影響。當 battery 越接近最低容量時，能再放出的電量應該越少。

本設計在程式中設定：

$$\gamma^{dis} = 0.60$$

$$\eta^{dis} = 0.90$$

其中，

γ^{dis} 是 SOC-dependent discharge ratio， η^{dis} 是 discharging efficiency。

Level 2 的放電限制式為：

$$Dis_{b,t} \leq \min (Dis_b^{max}, 0.60 \cdot 0.90 \cdot \max (0, SOC_{b,t}^{sd} - SOC_b^{min}))$$

此限制式表示 battery 的放電量同時受到兩個條件限制：

1. 不可以超過固定最大放電量 Dis_b^{max} 。
2. 不可以超過目前 SOC 與最低 SOC 之間可用電量所允許的放電量。

當

$SOC_{b,t}^{sd}$ 越接近 SOC_b^{min} 時， $SOC_{b,t}^{sd} - SOC_b^{min}$ 會越小，因此可放電量也會下降。若 $SOC_{b,t}^{sd}$ 已經低於或等於 SOC_b^{min} ，則 $\max (0, SOC_{b,t}^{sd} - SOC_b^{min}) = 0$ ，代表 battery 不能再放電。進一步考慮目前 SOC 狀態，因此 battery 越接近最低容量時，可放電量越小。

8. No simultaneous charging and discharging

Level 1 已有 battery 不可同時充電與放電的基本限制概念。

Level 2 重點

Level 2 在加入 efficiency、self-discharge、throughput limit、aging cost 與 SOC-dependent charge/discharge limit 後，仍會檢查同一 battery 在同一時間不可同時充放電：

$$Chg_{b,t} \cdot Dis_{b,t} = 0$$

若 battery

b 在時間 t 充電，則它不能同時放電；若 battery b 在時間 t 放電，則它不能同時充電。

演算法實作對應

在 `advanced_scheduler_plus.py` 中，battery 相關限制分別實作於 `apply_self_discharge()`、`soc_dependent_discharge_limit()`、`soc_dependent_charge_limit()` 與 `dispatch_advanced_energy()`。演算法會在每個時間點更新 SOC、throughput usage、battery charge/discharge amount 與 aging cost，並在 validation 中檢查 SOC update、SOC bound、SOC-dependent charge/discharge limit 與 no simultaneous charging/discharging。

放寬 Assumption 3：彈性市場機制

Level 1 假設市場價格已知，且不考慮 day-ahead sell commitment、售電承諾不足或違約成本。因此 Level 1 的市場收益主要只根據實際售電量計算：

$$R^{market} = \sum_{t \in T} \lambda_t Sell_t$$

其中，

λ_t 是時間 t 的市場價格， $Sell_t$ 是時間 t 實際售出到市場的電量。

也就是說，在 Level 1 中，系統只關心每個時間點最後實際賣了多少電，並沒有考慮「日前原本承諾要賣多少電」以及「如果實際售電量不足會不會產生 penalty」。

Level 2 中，本設計放寬此 assumption，加入以下市場機制：

1. day-ahead sell commitment
2. sell shortfall
3. cancellation penalty
4. market-aware battery dispatch rule

Energy balance and sell amount

每個時間點的實際售電量

$Sell_t$ 由 energy balance 推導而來：

$$Sell_t = \sum_{i \in I} P_{i,t} - \sum_{j \in J_p \cup J_s \cup J_a \cup J_{chg}} \sum_{i \in I} k_{j,i,t}$$

此式表示：時間

t 的總發電量扣掉供應外部 jobs 與 battery charging jobs 的電量後，剩下的電量就是可以售出到市場的電量。

其中：

符號	說明
$\sum_{i \in I} P_{i,t}$	時間 t 的總供電量
$\sum_{j \in J_p \cup J_s \cup J_a \cup J_{chg}} \sum_{i \in I} k_{j,i,t}$	時間 t 供應給 jobs 與 charging jobs 的總電量
$Sell_t$	時間 t 實際售出到市場的電量

在 Level 1 中，

$Sell_t$ 只用來計算 market revenue。Level 2 則會進一步拿 $Sell_t$ 與 day-ahead sell commitment 比較，以判斷是否產生 sell shortfall 與 cancellation penalty。

1. Day-ahead sell commitment

Level 1 中沒有 day-ahead sell commitment 的概念，只計算

$Sell_t$ 。

Level 2 放寬方式

Level 2 中加入 day-ahead sell commitment。Day-ahead sell commitment 指的是系統在日前規劃中，預期某個時間點可以承諾售出的電量。

本設計並沒有新增市場投標決策變數，而是使用 Level 1 的 thermal generator plan 作為 day-ahead baseline。也就是說，先看 Level 1 原本規劃 thermal generators 在時間

t 會發出多少電，再扣掉外部 jobs 在該時間點需要消耗的電量。如果扣掉外部需求後仍有剩餘，該剩餘量就視為時間 t 的 day-ahead sell commitment。

先定義時間

t 的外部 job demand：

$$D_t^{ext} = \sum_{j \in J_p \cup J_s \cup J_a} \sum_{i \in I} k_{j,i,t}$$

其中，

D_t^{ext} 表示時間 t 中 periodic、sporadic 與 aperiodic jobs 實際消耗的總電量。

這裡不包含 battery charging jobs，也就是不包含

J_{chg} ，因為 battery charging 屬於 VPP 內部儲能操作，不是外部用電需求。

接著，Level 2 的 day-ahead sell commitment 定義為：

$$S_t^{DA} = \max(0, \sum_{g \in I_g} P_{g,t}^{plan} - D_t^{ext})$$

其中：

符號	說明
$P_{g,t}^{plan}$	Level 1 中 thermal generator g 在時間 t 的計畫出力
D_t^{ext}	時間 t 的外部 job demand
S_t^{DA}	時間 t 的 day-ahead sell commitment

此式表示：若 Level 1 thermal generator plan 扣掉外部 job demand 後仍有剩餘，該剩餘電量就視為日前承諾可售出的電量；若扣除後小於 0，代表沒有可承諾售出的電量，因此

$S_t^{DA} = 0$ 。

例如，若時間

t 的 Level 1 thermal generator plan 為 70 MWh，外部 job demand 為 55 MWh，則：

$$S_t^{DA} = \max(0, 70 - 55) = 15 \text{ MWh}$$

代表系統在日前規劃中，預期時間

t 可以承諾售出 15 MWh。

2. Sell shortfall

Level 1 中不檢查實際售電量是否達成日前承諾，因為 Level 1 沒有

S_t^{DA} 。

Level 2 放寬方式

Level 2 中，advanced dispatch 完成後，系統會比較實際售電量

$Sell_t$ 與日前售電承諾 S_t^{DA} 。如果實際售電量低於日前承諾量，則產生 sell shortfall。

Sell shortfall 定義為：

$$Short_t = \max(0, S_t^{DA} - Sell_t)$$

其中，

$Short_t$ 表示時間 t 的售電承諾不足量。

如果實際售電量達成或超過日前承諾：

$$Sell_t \geq S_t^{DA}$$

則：

$$Short_t = 0$$

如果實際售電量低於日前承諾：

$$Sell_t < S_t^{DA}$$

則：

$$Short_t = S_t^{DA} - Sell_t$$

也就是說，

$Short_t$ 用來量化「原本承諾要賣的電量」與「實際賣出的電量」之間的差距。

3. Cancellation penalty

Level 1 中不考慮售電承諾不足造成的 penalty，因此不會因為售電量不足而增加 objective cost。

Level 2 放寬方式

Level 2 中，若實際售電量低於 day-ahead sell commitment，系統需要支付 cancellation penalty。

本設計在程式中設定：

$$\pi^{cancel} = 0.25$$

其中，

π^{cancel} 是 sell commitment penalty rate，代表 shortfall 會依市場價格的 25% 計算 penalty。

Level 2 的 cancellation penalty 定義為：

$$C^{cancel} = \sum_{t \in T} \pi^{cancel} \lambda_t Short_t$$

由於本設計設定

$\pi^{cancel} = 0.25$ ，因此可寫成：

$$C^{cancel} = \sum_{t \in T} 0.25 \lambda_t Short_t$$

此式表示：若時間

t 的實際售電量低於日前承諾，則不足的電量 $Short_t$ 會根據該時間點的市場價格 λ_t 與 penalty rate 0.25 轉換成 penalty cost。

例如，若某時間點：

$$S_t^{DA} = 20, \quad Sell_t = 15, \quad \lambda_t = 100$$

則 sell shortfall 為：

$$Short_t = \max(0, 20 - 15) = 5$$

該時間點的 cancellation penalty 為：

$$0.25 \times 100 \times 5 = 125$$

因此，Level 2 不只追求售電收益，也會考慮沒有達成日前售電承諾時的市場風險。

4. Market-aware battery dispatch rule

Level 1 中，市場收益主要根據實際剩餘電量計算，battery 使用簡化的低價充電、高價放電套利的方法。

Level 2 放寬方式

Level 2 中，battery dispatch 會受到市場價格影響。因此battery 不只是單純充放電，而是會根據市場價格判斷何時適合充電或放電。

本設計在程式中設定：

$$Q^{low} = 0.30$$

$$Q^{high} = 0.70$$

也就是使用 72 小時市場價格的第 30 百分位數作為低價門檻，第 70 百分位數作為高價門檻。

低價門檻與高價門檻定義為：

$$\lambda^{low} = Q_{0.30}(\lambda_t: t \in T)$$

$$\lambda^{high} = Q_{0.70}(\lambda_t: t \in T)$$

其中：

符號	說明
λ^{low}	低價門檻
λ^{high}	高價門檻
$Q_{0.30}$	第 30 百分位數
$Q_{0.70}$	第 70 百分位數

當市場價格低，或存在 renewable surplus 時，允許 battery charging：

$$Chg_{b,t} > 0 \Rightarrow \lambda_t \leq \lambda^{low} \text{ or } PVSurplus_t > 0$$

當市場價格高時，允許 battery discharging：

$$Dis_{b,t} > 0 \Rightarrow \lambda_t \geq \lambda^{high}$$

此規則表示：低價時段或 renewable surplus 時，系統可以將電存入 battery；高價時段，battery 可以放電支援外部 jobs，使其他發電量有更多機會轉為售電收益。

5. Level 2 objective 中的市場風險

Level 1 的 objective 可以表示為：

$$F = \alpha \sum_{j \in J_a} Miss_j + C^{thermal} - R^{market}$$

其中：

$$R^{market} = \sum_{t \in T} \lambda_t Sell_t$$

Level 2 加入 battery aging cost 與 sell commitment cancellation penalty :

$$F^{adv} = \alpha \sum_{j \in J_a} Miss_j + C^{thermal} + C^{age} + C^{cancel} - R^{market}$$

其中：

$$C^{cancel} = \sum_{t \in T} 0.25 \lambda_t Short_t$$

這代表 Level 2 中，售電收益不再只看

R^{market} ，也會受到 C^{cancel} 影響。如果 advanced dispatch 後的實際售電量低於日前承諾量，則 objective value 會因 cancellation penalty 而增加。

演算法實作對應

在 `advanced_scheduler_plus.py` 中，彈性市場機制主要實作於 `build_day_ahead_sell_commitment()`、`dispatch_advanced_energy()` 與 `evaluate_advanced()`。`build_day_ahead_sell_commitment()` 會根據 Level 1 的 thermal generator plan 與外部 job demand 建立每個時間點的 day-ahead sell commitment，也就是 S_t^{DA} 。

接著，`dispatch_advanced_energy()` 會在每個時間點重新計算實際售電量 `sell`，並檢查實際售電量是否低於 day-ahead sell commitment。若實際售電量不足，程式會記錄 `sell_commitment_shortfall_mwh`，並根據 `SELL_COMMITMENT_PENALTY_RATE = 0.25` 計算 `sell_commitment_penalty`。

此外，`dispatch_advanced_energy()` 也會使用 `LOW_PRICE_QUANTILE = 0.30` 與 `HIGH_PRICE_QUANTILE = 0.70` 設定市場價格門檻，作為 battery charging 與 discharging 的判斷依據。最後，`evaluate_advanced()` 會將 `sell_commitment_penalty` 納入 Level 2 objective，使售電承諾不足不只是被記錄，也會實際反映在 objective cost 中。

與 Level 1 的整體差異整理

放寬項目	Level 1	Level 2 advanced scheduler
Renewable output	使用 forecast output	使用 actual renewable availability
Renewable constraint	$P_{r,t} \leq Cap_r F_{r,t}$	$P_{r,t} \leq Cap_r A_{r,t}$
Forecast uncertainty	不考慮	使用 $\theta_{r,t}$ 建立 actual output
Battery SOC update	理想或簡化更新	加入 self-discharge、charging efficiency、discharging efficiency
Battery aging	不考慮	加入 C^{age}
Battery throughput	不限制總使用量	加入 $\sum_t (Chg_{b,t} + Dis_{b,t}) \leq L_b$
Charge/discharge power	固定上限	SOC-dependent charge/discharge limit
Market revenue	只計算 R^{market}	同時計算 R^{market} 與 C^{cancel}
Sell commitment	不考慮	加入 S_t^{DA} 與 $Short_t$
Objective	$\alpha Miss + C^{thermal} - R^{market}$	$\alpha Miss + C^{thermal} + C^{age} + C^{cancel} - R^m$
Dispatch method	static schedule / simplified energy dispatch	rolling corrective dispatch

設計相關參數表

程式參數	數學符號	數值	說明
CHARGE_EFFICIENCY	η^{ch}	0.95	battery charging efficiency
DISCHARGE_EFFICIENCY	η^{dis}	0.90	battery discharging efficiency
BATTERY_AGING_COST_PER_MWH	c^{age}	5.0	每 MWh battery throughput 的 aging cost
BATTERY_DAILY_THROUGHPUT_RATIO	R^{cycle}	1.20	battery throughput limit 的容量比例
BATTERY_SELF_DISCHARGE_RATE	ρ_b	0.0005	battery 每小時 self-discharge rate
SOC_DEPENDENT_CHARGE_RATIO	γ^{ch}	0.60	SOC-dependent charge limit ratio
SOC_DEPENDENT_DISCHARGE_RATIO	γ^{dis}	0.60	SOC-dependent discharge limit ratio
SELL_COMMITMENT_PENALTY_RATE	π^{cancel}	0.25	sell commitment shortfall penalty rate
LOW_PRICE_QUANTILE	$Q_{0.30}$	0.30	定義 low price threshold
HIGH_PRICE_QUANTILE	$Q_{0.70}$	0.70	定義 high price threshold
EPS	ϵ	10^{-6}	浮點數誤差容忍值

實作於演算法中的對應關係

放寬 assumption	對應函式 / 程式邏輯	實作內容
再生能源不確定性	uncertainty_factor()	產生 $\theta_{r,t}$
再生能源不確定性	build_actual_renewable()	計算 $A_{r,t}$ 與 actual available MWh
actual renewable dispatch	dispatch_advanced_energy()	使用 actual renewable 供應 jobs、charging 或 sell
self-discharge	apply_self_discharge()	更新 $SOC'_{b,t}$
SOC-dependent discharge	soc_dependent_discharge_limit()	計算 battery 可放電上限
SOC-dependent charge	soc_dependent_charge_limit()	計算 battery 可充電上限
battery aging cost	dispatch_advanced_energy()	根據 charging/discharging throughput 累加 aging cost
battery throughput limit	dispatch_advanced_energy()	追蹤 throughput_used 並限制 battery usage
day-ahead sell commitment	build_day_ahead_sell_commitment()	建立 S_t^{DA}
sell shortfall penalty	dispatch_advanced_energy()	計算 $Short_t$ 與 C^{cancel}
objective evaluation	evaluate_advanced()	計算 $C^{thermal}$ 、 C^{age} 、 C^{cancel} 、 R^{market} 與 F^{adv}
constraint validation	validate_advanced_schedule()	檢查 renewable bound、SOC update、SOC bound、energy balance、charge/discharge consistency

日前提程

本小組在 Level 1 中採用 **reserve-based static scheduling** 作為日前提程策略。排程器會先在 72 小時的排程範圍內建立一份固定的 day-ahead schedule，並在排程過程中預留部分可用容量，讓後續 sporadic jobs 到達時仍有機會被插入。

本方法的核心想法是：

先確保 hard real-time jobs 的可行性，再利用剩餘容量服務 soft real-time jobs 與市場售電。

因此，排程順序依照任務重要性分為三層：

Periodic jobs > Sporadic jobs > Aperiodic jobs

其中，periodic jobs 與 sporadic jobs 屬於 hard-deadline jobs，必須在 deadline 前完成；
aperiodic jobs 屬於 soft-deadline jobs，系統會盡量在 deadline 前完成，但若資源不足，則允許延後或記錄 miss。

演算法執行邏輯

Level 1 的日前排程流程如下：

1. 展開 periodic jobs
2. 排程 periodic jobs，並預留 sporadic reserve
3. 對 sporadic jobs 執行 acceptance test
4. 排程 aperiodic waiting queue
5. 完成能源平衡與供電來源分配

此設計並不是在每個 sporadic job 到達後重新安排所有 jobs，而是在既有 periodic schedule 不被移動的前提下，檢查剩餘容量是否足以插入新的 sporadic job。

也就是說，Level 1 的排程器會先建立一份穩定的日前排程，然後只在剩餘空檔中嘗試插入動態到達的 sporadic jobs，這樣可以避免 sporadic jobs 到達後破壞原本已經排好的 hard-deadline jobs。

可用容量與 reserve 的定義

在每一個時間點

t ，系統會根據傳統機組的計畫輸出計算可用容量：

$$Capacity_t = \sum_{g \in G} P_{g,t}$$

其中，

$P_{g,t}$ 表示 generator g 在時間 t 的輸出量。

Level 1 中，傳統機組的輸出量不是一開始就直接設為最大輸出，而是受到 ramp-up rate 與最大輸出量限制，ramp-up rate 表示 generator 每個 time slot 最多可以增加多少輸出，因此 generator 需要隨時間逐步升載，不能在初始時段就瞬間提供最大發電量。

其輸出量定義為：

$$P_{g,t} = \min(P_g^{max}, RampUp_g \times t)$$

其中，

P_g^{max} 是 generator g 的最大輸出量， $RampUp_g \times t$ 則表示在時間 t 依照 ramp-up rate 最多可以升載到的輸出量。系統取兩者中的較小值，代表 generator 的輸出不能超過最大輸出，也不能超過目前時間點按照 ramp-up 限制可達到的輸出。

因此，在排程前期，generator 可能尚未升載到最大輸出，可用容量會比較小；隨著時間增加，generator 可提供的容量才會逐步提高，直到達到最大輸出為止。

當排程器要將 job

j 放入時間點 t 時，會檢查該時間點是否仍有足夠容量。若 job j 的能源需求為 w_j ，則可行性條件為：

$$Load_t + w_j \leq Capacity_t - Reserve_t$$

其中：

符號	說明
$Load_t$	時間 t 已經排入的工作負載
w_j	job j 在單一時間 slot 的能源需求
$Capacity_t$	時間 t 的可用發電容量
$Reserve_t$	時間 t 預留給後續 jobs 的容量

因此，reserve 的作用是讓排程器在安排前面的任務時，不要立刻把所有容量用完，保留後續 sporadic jobs 或 aperiodic jobs 可以使用的空檔與能源餘裕。

Periodic jobs 的排程與 sporadic reserve

Periodic jobs 是系統會最先排程 periodic jobs。

在排程 periodic jobs 時，系統會優先保留：

```
SPORADIC_RESERVE_MWH = 20.0
```

也就是說，當排程器尋找 periodic job 的可行時段時，會盡量讓每個時間點留下 20 MWh 的容量給未來可能到達的 sporadic jobs。

此時的可行性條件為：

$$Load_t + w_j \leq Capacity_t - 20$$

此設計的目的為：

在 periodic jobs 排程階段，避免固定任務佔滿所有可用容量，提前替 sporadic jobs 保留插入空間。

不過，這個 reserve 並不是絕對不可使用的硬限制。若某個 periodic job 在保留 20 MWh 的情況下無法找到可行時段，系統會取消 reserve，改用：

```
reserve_after = 0.0
```

重新嘗試排程。

也就是說，periodic jobs 的排程邏輯為：

```

先嘗試保留 20 MWh 給 sporadic jobs
↓
若 periodic job 可排入，則保留成功
↓
若 periodic job 無法排入，取消 reserve 後重新嘗試
↓
若仍無法排入，代表該 periodic job 無法被排程

```

這樣設計的原因是 periodic jobs 系統不能為了保留 sporadic reserve 而犧牲 periodic jobs 的可行性。

因此，Level 1 的 sporadic reserve 屬於 **soft reservation**：

能保留時就保留；但若 hard-deadline periodic jobs 無法排入，則優先滿足 periodic jobs。

Sporadic jobs 到達後如何使用 reserve

Sporadic jobs 是動態到達的 hard-deadline jobs。它們在日前排程階段不一定已經完全確定，但一旦到達後，系統必須決定是否接受。

當 sporadic job 到達後，系統會執行 acceptance test。其流程如下：

1. 讀取 sporadic job 的 release time、deadline、execution time 與 energy demand
2. 在既有 schedule 中尋找可行時段
3. 若找到足夠時段，接受該 sporadic job
4. 若找不到足夠時段，拒絕該 sporadic job

此時，sporadic jobs 會使用前面 periodic scheduling 階段保留下來的容量。換句話說，20 MWh 的 sporadic reserve 並不是指定給某一個特定 job，而是保留在時間軸上的可用空檔與能源餘裕。

當 sporadic job 到達時，系統會根據該 job 的 release time 與 deadline，在可用時間範圍內尋找 feasible slots。若存在足夠容量，使得：

$$Load_t + w_j \leq Capacity_t$$

則該 sporadic job 可以被接受並插入 schedule。

若找不到可行時段，系統不會移動已經排好的 periodic jobs，也不會進行全域重新排程，而是直接拒絕該 sporadic job。

因此，sporadic acceptance test 的本質是：

在不破壞既有 hard-deadline schedule 的前提下，檢查日前保留的空檔與能源餘裕是否足以容納新的 sporadic job。

Aperiodic reserve 的處理

Aperiodic jobs 屬於 soft-deadline jobs，因此優先級低於 periodic jobs 與 sporadic jobs。系統會在 hard-deadline jobs 排完後，再處理 aperiodic waiting queue。

為了避免 sporadic jobs 完全佔滿剩餘容量，系統在插入 sporadic jobs 時，會先嘗試保留：

$$\text{APERIODIC_RESERVE_MWH} = 15.0$$

也就是說，當系統測試某個 sporadic job 是否可以被接受時，會先檢查在保留 15 MWh 給 aperiodic jobs 的情況下，sporadic job 是否仍然可以排入：

$$Load_t + w_j \leq Capacity_t - 15$$

若可以，代表系統接受 sporadic job 的同時，也保留部分容量給後續 aperiodic jobs 使用。

若不可以，系統會取消 aperiodic reserve，再次檢查：

$$Load_t + w_j \leq Capacity_t$$

如果取消 reserve 後可以排入，系統仍會接受該 sporadic job。這代表在 hard-deadline sporadic jobs 與 soft-deadline aperiodic jobs 之間發生衝突時，系統會優先滿足 sporadic jobs。

因此，aperiodic reserve 的判斷邏輯如下：

```

先嘗試保留 15 MWh 給 aperiodic jobs
↓
若 sporadic job 仍可排入，則保留成功
↓
若 sporadic job 無法排入，取消 aperiodic reserve
↓
若取消 reserve 後可排入，接受 sporadic job
↓
若仍無法排入，拒絕 sporadic job

```

此設計反映了 Level 1 的優先順序：

$$Periodic > Sporadic > Aperiodic$$

也就是說，系統會盡量保留資源給 aperiodic jobs，但當 hard-deadline sporadic jobs 需要使用這些資源時，系統會優先接受 sporadic jobs。

Non-preemptive jobs 的可行時段選擇

若 job 不可搶佔，也就是：

$$\text{preempt} == 0$$

則該 job 必須被安排在一段連續的時間區塊中。

系統會從 release time 開始掃描到 deadline，檢查是否存在長度等於 execution time 的連續 block，且該 block 內每一個時間點都滿足容量限制：

$$Load_t + w_j \leq Capacity_t - Reserve_t$$

若有多個可行 block，系統會根據以下因素選擇較好的區塊：

1. 儘量不跨越 frame；
2. 儘量選擇市場價格較低的時段；
3. 若前兩者相同，選擇較早的時段。

因此，non-preemptive jobs 的排程策略可以整理為：

找一段連續、可行、低電價，且盡量不跨 frame 的執行區塊。

其中，frame size 設定為：

$$F = 3$$

代表系統會以 3 小時為一個 frame，盡量避免 non-preemptive jobs 橫跨不同 frame，以降低排程碎片化與後續插入 sporadic jobs 的困難度。

Preemptive jobs 的可行時段選擇

若 job 可搶佔，也就是：

```
preempt == 1
```

則該 job 不需要被安排在連續時段中，可以被切割到不同時間點執行。

系統會先在 release time 到 deadline 之間找出所有 feasible slots，接著依照以下順序排序：

1. 市場價格較低者優先；
2. frame ID 較早者優先；
3. 時間較早者優先。

最後，系統會選出前

e_j 個時段，其中 e_j 為該 job 的 execution time。

因此，preemptive jobs 的排程策略可以整理為：

在 deadline 前找出足夠的可行時段，並優先使用低電價時段執行。

這樣做的原因是，若工作安排在高電價時段，該時段的能源就不能拿去售電，可能降低市場收益。因此 Level 1 會在滿足 deadline feasibility 的前提下，盡量把可搶佔任務安排到低電價時段。

再生能源在 Level 1 中的處理

Level 1 中的 reserve 主要是針對 job scheduling 的可用容量設計，也就是根據傳統機組可提供的容量，決定 jobs 是否能排入。

再生能源在 Level 1 中被視為不可調度式資源。系統不主動控制再生能源輸出，也不把再生能源作為 hard-deadline jobs 的主要 reserve 來源。完成 job scheduling 與傳統機組供電分配後，系統會依照 renewable forecast 計算每個時間點的再生能源輸出：

$$P_{r,t} = Capacity_r \times Forecast_{r,t}$$

其中：

符號	說明
$P_{r,t}$	renewable generator r 在時間 t 的發電量
$Capacity_r$	renewable generator r 的裝置容量
$Forecast_{r,t}$	時間 t 的再生能源預測係數

在 Level 1 中，再生能源主要被視為額外可出售的能源來源，而不是用來改變 job schedule 的動態供電來源。因此，完成 jobs 的供電分配後，forecast renewable energy 會被加入市場售電量。

若程式中仍出現 `sell_all_solar()` 這類名稱，這只是早期命名留下來的函式名稱；在報告中應統一描述為 **renewable energy** 或 **再生能源**，而不是 solar energy。

小結

Level 1 的日前排程策略可以總結為 **reserve-based static scheduling**。系統先排入 periodic jobs，並嘗試保留 20 MWh 的 sporadic reserve，當 sporadic jobs 到達後，系統在不移動既有 hard-deadline jobs 的前提下執行 acceptance test，接著再使用剩餘容量處理 aperiodic jobs。

本方法的重點不是每次有新任務到達就重新最佳化整份 schedule，而是透過日前保留空檔與能源餘裕，提高後續 sporadic jobs 被接受的可能性，同時維持既有 hard-deadline schedule 的穩定性。

整體而言，本方法的設計取捨為：

犧牲部分即時使用容量與售電彈性，換取 sporadic jobs 的插入空間與 hard-deadline jobs 的排程穩定性。

進階動態排程方法 [Only Level 2]

方法概念：Rolling Corrective Dispatch

在 Level 2 中，本設計採用 **Rolling Corrective Dispatch** 作為進階動態排程方法，並非每次都重新安排所有 jobs 的執行時間，而是在 Level 1 已建立的 72 小時日前固定排程基礎上，根據即時狀態更新新能源分配。

Level 1 中，periodic jobs、accepted sporadic jobs 與 aperiodic jobs 已經被安排到特定時間點。Level 2 的 advanced scheduler 會保留這些 hard-deadline jobs 的時間安排，並在排程執行過程中根據以下動態資訊修正每小時的 energy dispatch：

1. actual renewable availability；
2. battery SOC；
3. battery throughput usage；
4. market price；

5. day-ahead sell commitment ;
6. real-time sell shortfall 。

因此，本方法的核心是 在不破壞既有 **hard-deadline schedule** 的前提下，動態修正能源來源、**battery charge/discharge** 行為與售電量。

選擇此方法的理由

選擇 Rolling Corrective Dispatch 的主要理由有四點。

第一，Level 2 放寬了 renewable output 完全準確的 assumption。由於 actual renewable output 可能高於或低於 forecast output，若仍使用 Level 1 的 static dispatch，可能會高估或低估每個時間點的可用 renewable energy。因此，advanced scheduler 需要在排程過程中根據 actual renewable availability 修正能源分配。

第二，Level 2 加入 battery realism。Battery 不再是理想化儲能設備，而是受到 charging efficiency、discharging efficiency、self-discharge、aging cost、throughput limit 與 SOC-dependent power limit 影響。因此，battery 的使用必須根據即時 SOC 與剩餘 throughput budget 動態判斷，不能在日前排程時一次固定所有充放電行為。

第三，Level 2 加入 day-ahead sell commitment 與 cancellation penalty。系統不只需要考慮實際售電收益，也需要考慮若 real-time sell 低於日前承諾售電量時產生的 penalty。因此，advanced scheduler 必須在 jobs demand、battery charging、market selling 與 commitment shortfall 之間做即時權衡。

第四，因為我們在 Level 1 的時候主要想達到的目標是最小化 miss rate，到了 Level 2 我們則是不只想要保持我們原本最小化 miss rate 的目標，還想要盡可能的做到最小化傳統機組發電成本和最大化售電收益

更新時機

本方法使用 rolling window 進行動態更新，設定：

$$\Delta = 3 \text{ hours}$$

也就是每 3 小時進行一次 rolling update。

由於總排程範圍為：

$$H = 72$$

因此 72 小時內最多會進行：

$$\frac{72}{3} = 24$$

次 rolling update。

在每次 rolling update 中，系統會觀察或更新以下資訊：

更新資訊	說明
actual renewable availability	根據 forecast ratio 與 uncertainty factor 計算實際可用 renewable energy
battery SOC	根據 self-discharge、charge、discharge 更新 SOC
throughput usage	記錄 battery 累積充放電量，避免過度使用
market price status	判斷目前是否為低價、高價或一般時段
day-ahead sell commitment	檢查目前實際售電量是否低於日前售電承諾
sell shortfall penalty	若實際售電不足，計算 cancellation penalty

整體流程

Rolling Corrective Dispatch 的流程可以分為三個階段。

1. Day-ahead schedule initialization

首先，系統使用 Level 1 scheduler 建立 72 小時日前固定排程。此階段會完成：

1. periodic jobs expansion；
2. periodic jobs scheduling；
3. sporadic jobs acceptance test；
4. accepted sporadic jobs insertion；
5. aperiodic waiting queue scheduling；
6. Level 1 thermal generator plan；
7. 初始 market sell amount 計算。

然後Level 2 會根據 Level 1 的 thermal generator plan 與 external job demand 建立 day-ahead sell commitment：

$$S_t^{DA}$$

其中，

S_t^{DA} 表示時間 t 的日前承諾售電量。

2. Rolling update

每 3 小時進入一次 rolling update window，設 rolling window 為：

$$W_m = m, m + 1, m + 2$$

其中

m 為 rolling window 的起始時間。

在每個 rolling window 中，advanced scheduler 會讀取該 window 內的 actual renewable availability，並逐小時修正 energy dispatch。

3. Hourly corrective dispatch

在每個時間點

t ，advanced scheduler 依序執行以下步驟。

Step 1：更新 battery self-discharge

在進行 charge 或 discharge 之前，先更新 battery 的 self-discharge：

$$SOC' * b, t = \max(SOC_b^{min}, (1 - \rho_b)SOC * b, t - 1)$$

其中，

$SOC'_{b,t}$ 表示 battery b 在時間 t 經過 self-discharge 後的 SOC。

Step 2：使用 actual renewable 優先供應 jobs

接著，系統根據 actual renewable availability 決定可用 renewable energy：

$$Available^{actual} * r, t = Cap_r A * r, t$$

其中：

$$A_{r,t} = \min(1, \max(0, F_{r,t} \theta_{r,t}))$$

若 renewable energy 可用，advanced scheduler 會優先使用 renewable 供應當前時間點的 external jobs demand。此設計可以降低傳統機組供電需求，並減少 thermal generation cost。

Step 3：高價時段考慮 battery discharging

若 renewable energy 不足以供應當前 jobs demand，且目前市場價格屬於高價時段，advanced scheduler 會考慮使用 battery discharge 補足需求。

高價門檻定義為：

$$\lambda^{high} = Q_{0.70}(\lambda_t \in T)$$

當：

$$\lambda_t \geq \lambda^{high}$$

時，系統允許 battery discharging。

Battery discharge 必須同時滿足：

1. 不超過剩餘 job demand；

2. 不超過 battery 每小時最大放電量；
3. 不超過 SOC-dependent discharge limit；
4. 不超過剩餘 throughput budget；
5. 不使 SOC 低於 SOC_b^{min} 。

因此，battery 不是無限制使用，而是作為 renewable shortage 或高價時段的 corrective resource。

Step 4：使用 thermal generator 補足剩餘需求

若 renewable 與 battery discharge 仍不足以供應所有 jobs，則使用 thermal generator 補足剩餘需求。

此設計讓 thermal generator 不會過度使用≤但可以確保 hard-deadline jobs 的能源需求被滿足。

Step 5：低價或 renewable surplus 時考慮 battery charging

在 jobs demand 被供應後，若仍有 renewable surplus≤或目前市場價格屬於低價時段，advanced scheduler 會考慮 battery charging。

低價門檻定義為：

$$\lambda^{low} = Q_{0.30}(\lambda_t \in T)$$

當：

$$\lambda_t \leq \lambda^{low}$$

或 renewable surplus 大於 0 時，系統允許 battery charging。

Battery charging 必須同時滿足：

1. 不超過 battery 每小時最大充電量；
2. 不超過 SOC-dependent charge limit；
3. 不超過剩餘 throughput budget；
4. 不使 SOC 超過 SOC_b^{max} 。

此規則代表系統會在低價或 renewable surplus 時將能源存入 battery，以提高後續高價時段或 renewable shortage 時的調度彈性。

Step 6：計算實際售電量

完成 jobs supply 與 battery charging 後，剩餘可用電量會出售到市場：

$$Sell_t = \sum_{i \in I} P_{i,t} - \sum_{j \in J} \sum_{i \in I} k_{j,i,t}$$

其中，

$Sell_t$ 表示時間 t 的實際售電量。

若在供應完當下 jobs 需求之後，還剩下的多餘電量沒有被 jobs 或 battery charging 使用，則會轉為 market sell，以增加售電收益。

Step 7：檢查 day-ahead sell commitment shortfall

最後，advanced scheduler 會檢查實際售電量是否低於 day-ahead sell commitment：

$$Short_t = \max(0, S_t^{DA} - Sell_t)$$

若

$Short_t > 0$ ，代表 real-time sell 無法滿足日前售電承諾，因此會產生 cancellation penalty：

$$C^{cancel} = \sum_{t \in T} 0.25\lambda_t Short_t$$

此設計使售電承諾不足不只是被記錄，也會反映在 Level 2 objective value 中。

判斷規則整理

本方法的主要判斷規則如下。

判斷項目	規則	設計目的
Hard-deadline preservation	periodic jobs 與 accepted sporadic jobs 不移動	避免破壞已通過 acceptance test 的 hard-deadline feasibility
Renewable priority	actual renewable 優先供應 jobs	降低 thermal generation cost
Renewable shortage correction	renewable 不足時，先考慮 battery，再使用 thermal	提高 renewable uncertainty 下的供電可行性
Battery charging	低價或 renewable surplus 時允許 charging	儲存便宜能源或多餘 renewable
Battery discharging	高價時段允許 discharging	降低高價時段 thermal usage，增加市場彈性
Throughput control	每次 charge/discharge 都消耗 throughput budget	避免 battery 被過度使用
Aging cost accounting	battery throughput 會產生 aging cost	避免將 battery 視為免費資源
Sell commitment check	若 $Sell_t < S_t^{DA}$ ，計算 shortfall penalty	反映市場承諾不足的風險

對重排頻率的影響

本方法的重排頻率設定為 **每 3 小時更新一次** ≤ 相較於每個時間點都重新排程，3 小時 rolling window 可以在即時性與穩定性之間取得平衡。

若重排太頻繁，例如每 1 小時重新計算一次，系統雖然能更即時反映再生能源變化與市場價格，但會造成較高的計算負擔，也可能導致能源分配結果過度震盪。

相反地，若重排間隔太長，例如每 6 小時或 12 小時才更新一次，系統雖然較穩定，但可能無法及時修正再生能源預測誤差，導致電池調度、傳統機組補足與售電決策較不精準。

因此，本方法選擇 3 小時作為 rolling update interval，使系統能夠定期根據即時資訊修正能源分配，同時避免過度頻繁的全域重排。

此外，本方法主要重排的是能源分配量：

$$k_{j,i,t}^{adv}$$

也就是 job 在不同能源來源之間的分配量，而不是任意改動 job 的執行時間。對於 periodic jobs 與已接受的 sporadic jobs，其 hard-deadline schedule 會盡量保留：

$$x_{j,t}^{adv} = x_{j,t}^{L1}$$

因此可以降低重排對 hard real-time feasibility 的破壞。

對計算成本的影響

本方法的計算成本會比 Level 1 靜態排程方法更高，因為 Level 2 額外考慮了多項動態因素，包括：

- 實際再生能源可用量與預測值之間的差異；
- 電池充放電效率；
- 電池 self-discharge；
- SOC-dependent charge/discharge limit；
- battery aging cost；
- day-ahead sell commitment cancellation penalty；
- 每 3 小時一次的 rolling corrective dispatch。

不過，本方法並不是每次都重新求解完整排程，而是保留原本 hard-deadline jobs 的排程結果，只修正能源 dispatch。因此它的計算成本主要來自能源分配、電池 SOC 更新、售電量計算與 penalty 計算，而不是重新安排所有 jobs 的執行時間。

因此，本方法的計算成本介於以下兩者之間：

Level 1 static scheduling < rolling corrective dispatch < full rescheduling at every time slot

對 sporadic job 接納率的影響

在目前實作中，sporadic job 的 accept / reject 主要仍由 Level 1 acceptance test 決定。Level 2 rolling corrective dispatch 不會重新移動已排定的 hard-deadline jobs，也不會重新對已 rejected 的 sporadic jobs 執行新的 acceptance test。

因此，本方法對 sporadic job 接納率的直接影響較有限：

$$AcceptanceRate^{L2} * sporadic * * \approx * * AcceptanceRate^{L1} * sporadic$$

但是，Level 2 對 sporadic jobs 仍有間接幫助。

1. 若 actual renewable output 低於 forecast，advanced scheduler 可以透過 battery discharge 或 thermal generator 補足能源，使已接受的 sporadic jobs 在 renewable uncertainty 下仍較容易維持能源可行性。
2. 若未來進一步將 rolling update 後的 SOC、renewable surplus 與 reserve 狀態回饋到 acceptance test，則 Level 2 可能提高 sporadic job acceptance rate。因為系統可以利用即時 renewable surplus 或 battery reserve，接受部分在 static schedule 下原本可能被拒絕的 sporadic jobs。

因此，目前本方法對 sporadic jobs 的接納率影響較小，主要效果是提高已接受 sporadic jobs 在 real-time uncertainty 下的供電穩定性與可行性。

三個目標函式之間的權衡關係

1. f_1 與 f_2 的權衡：miss penalty vs. generation cost

因為miss penalty的成本很高，所以本方法會優先避免 aperiodic miss，因為 miss penalty 被設定得很高。這樣可以提升服務品質，但代價是可能增加傳統機組發電成本。

2. f_1 與 f_3 的權衡：job completion vs. market revenue

f_3 是售電收益的負值，因此售電越多， f_3 越小，整體目標函式越好。

然而，能源若被拿去售電，就可能無法用來執行 jobs。當系統有額外能源時，需要在以下兩者之間取捨：

1. 將能源用於完成 aperiodic jobs，降低 f_1 ；
2. 將能源賣到市場，提高 revenue，使 f_3 下降。

本方法會先滿足 job execution，再利用剩餘能源售電，以避免為了提高短期收益而犧牲 real-time performance。

3. f_2 與 f_3 的權衡：generation cost vs. market revenue

f_2 希望降低傳統機組發電成本，而 f_3 希望提高售電收益。這兩者之間存在明顯 trade-off。

若市場價格

λ_t 很高，系統可能會傾向增加可售電量，以提高 market revenue。然而，若為了售電而增加傳統機組輸出，則 f_2 也會增加。因此，系統需要判斷額外售電收益是否大於額外發電成本

若：

$$\lambda_t \cdot \Delta Sell_t > \Delta f_2$$

則增加發電並售電可能是有利的；反之，若市場價格低，則不值得為了售電而提高傳統機組輸出。

因此，本方法會在高電價時段更重視售電收益，在低電價時段則更傾向降低發電成本或將多餘再生能源存入電池。

Level 2 進階目標函式

在 Level 2 中，系統進一步加入 battery aging cost 與 sell commitment cancellation penalty，因此進階目標函式可表示為：

$$F^{adv} = \alpha f_1 + C^{thermal} + C^{age} + C^{cancel} - R^{market}$$

其中：

項目	說明
αf_1	aperiodic soft-deadline miss penalty
$C^{thermal}$	傳統機組發電成本
C^{age}	battery aging cost
C^{cancel}	day-ahead sell commitment shortfall penalty
R^{market}	市場售電收益

加入 battery aging cost 後，系統不會無限制地頻繁充放電，因為每次電池 throughput 都會增加

C^{age} 。因此，電池會被保留給較有價值的時段，例如再生能源不足、job demand 較高或市場價格較高的時段。

加入 sell commitment cancellation penalty 後，系統也不能只根據即時價格自由調整售電量。如果實際售電量低於 day-ahead commitment，會產生 shortfall penalty。因此系統需要在「保留能源給 jobs」與「滿足售電承諾」之間取得平衡。

預期整體影響總結

分析面向	預期影響
重排頻率	每 3 小時 rolling update，能反映即時資訊，但避免過度頻繁重排
計算成本	高於 Level 1，但低於每個 time slot 全域重新排程
sporadic job 接納率	不一定提高，因為仍以 hard-deadline feasibility 為優先
post-acceptance violation	預期降低或維持 0
aperiodic miss	因 α 很高，系統傾向避免 miss
傳統機組成本	可能上升，因為系統可能使用 thermal generation 來避免 miss 或補足 renewable shortage
售電收益	在高價時段提高，但會受到 job demand、battery SOC 與 commitment constraint 限制
電池使用	不會無限制充放電，因為受到 aging cost、throughput limit 與 SOC-dependent limit 限制
系統穩定性	提高，因為 hard-deadline schedule 被保留，只修正能源 dispatch

總結

本方法透過 **3 小時 rolling corrective dispatch**，在不破壞 hard-deadline job schedule 的前提下，根據實際再生能源、電池狀態與市場價格修正能源分配。

主要取捨放在：

優先維持 hard-deadline schedule 的穩定性與 real-time energy feasibility，再利用 renewable、battery 與 market-aware dispatch 改善成本與收益。

設計重點不是追求單一目標最大化，而是在 renewable uncertainty、battery realism 與 market risk 下，提供一個計算成本較低且穩定性較高的動態修正策略。

效能分析

日前排程

此處用助教提供的 aperiodic_n_sporadic範例.json 插入input運行整個程式，並分析其效能。

1. 系統效能指標總覽

下表彙整了本虛擬電廠排程系統在時間效能、動態准入、經濟效益及限制式合規性等維度之核心評估數據：

評估維度	核心指標 (Metrics)	實驗量測值	系統效能表現評述
時間與任務效能 (Timing & Tasks)	硬時限錯失率 (Hard Miss Rate)	0.0%	在高工作負載密度下，成功達成 硬/軟時限零逾期之排程。
	軟時限錯失率 (Soft Miss Rate)	0.0%	
	平均逾期時間 (Avg Tardiness)	0.0 hr	
	最大相對回應時間 (Max Response Time)	13.0 hr	
		1.65 hr	
	完成時間抖動率 (Jitter)		
動態准入控制 (Acceptance Test)	突發任務總數 (Sporadic Jobs)	2	Acceptance Test 在一定程度上追求收益 (工作優先分配在低電價時段)， 具硬實力(任務接受率高) 且未對既有排程造成擾動。
	突發任務接受率 (Acceptance Rate)	100.0%	
	接受後違規率 (Post-Violation Rate)	0.0%	
	突發任務價值率 (Value Rate)	1.0	

評估維度	核心指標 (Metrics)	實驗量測值	系統效能表現評述
經濟與優化效益 (Economics)	傳統機組總燃料成本 (Generator Cost)	\$588,870.0	在優先考慮能完成任務的前提下盡量 將營運成本壓低。
	電力市場售電收益 (Market Revenue)	\$551,766.2	
	最終目標函數值 (Objective Value)	\$37,103.8	
限制式合規性 (Constraints)	任務層級違規數 (Job Violations)	0	未違反要求的 23 項限制式， 排程完備，具可執行性。
	每小時物理約束違規數 (Hourly Violations)	0	
	總違規計數 (Total Violations)	0	

2. 時間與排程效能剖析 (Timing & Scheduling Performance)

本實驗所生成的週期性任務集包含 7 個獨立 Task，展開後在 72 小時內共計產生 34 個 Periodic Jobs。該任務集的總工作負載密度 (Workload Density) 達 (94.5%)。在如此負載下，排程器的表現如下：

2-1. 軟時限零錯失 (0% Miss Rate)：

實驗結果顯示，硬時限與軟時限任務的錯失率皆為 **0.0%**，平均逾期時間 (Average Tardiness) 為 **0.0 小時**。這印證了「第一階段：日前固定排程」中，基於合法 Frame Size (

$f = 3$) 的時間框架排程演算法具備一定的確定性，能有效在日前階段鎖定週期性任務的執行空間。

2-2. 回應時間與抖動 (Response Time & Jitter)：

系統整體的平均回應時間為 **5.03 小時**，而週期性任務的平均回應時間為 **4.13 小時**。完成時間抖動率 (Jitter) 控制在 **1.65 小時**。此數據反映出排程器並未盲目延遲任務，而是在時限寬裕度內進行了有效的分佈。

3. 動態准入控制與突發任務效能 (Dynamic Acceptance Control)

在動態即時排程測試中，系統面臨 Sporadic Jobs 的即時釋放。排程器採用了結合「時間窗連續性」與「剩餘電能容量天花板」的雙重 Acceptance Test 策略：

3-1. 盡可能多的追求 Sporadic task Accepted：

在測試的 2 個突發任務中，系統成功全數接受 (Acceptance Rate = 100%)，使得 (Sporadic Value Rate) 達到 1.0。

3-2. 0% Post-Acceptance Violation：

動態排程忌諱為了接新任務而搞垮舊系統。本次結果的接受後違規率 (Post-Acceptance Violation Rate) 為 0.0%。說明准入測試的 candidate_times 探測機制具相當精準性，新任務的插入沒有侵害到既有週期性任務與硬時限任務的資源安全性。

4. 經濟效益與能量平衡優化分析 (Economic & Financial Optimization)

本專題的終極優化目標函數值 (Objective Value) 計算公式為：

$$\text{Objective Value} = \text{Generator Cost} - \text{Market Revenue} + \text{Miss Penalties}$$

4.1 利潤分析：

傳統機組發電成本為 \$588,870.0，而市場售電收益為 \$551,766.2，使得最終淨支出為 \$37,103.8。

4.2 市場經濟套利行為分析：

任務排程的部分，scheduler 會在所有可行時間中，優先把任務排在電價相對低的時間段，間接使整體利潤增加。

至於發電邏輯的部分，雖然本研究在工作排程階段利用市場電價作為時段選擇依據，並於儲能管理階段採用價格導向之電池充放電策略，但傳統機組發電規劃仍採固定 ramp-up 之保守出力模型。其輸出僅受機組最大出力與爬升率限制，不受即時市場價格或負載需求影響。因此本研究之 thermal dispatch 著重於可行性與供電可靠度，而非經濟最佳化。

5. 限制式合規性結論 (Constraint Compliance)

虛擬電廠實務上物理連續性限制較為嚴苛。本系統在經歷獨立evaluator 掃描 72 小時、共 23 項物理與數學約束（包含傳統機組升降載率、最低/最高出力、儲能設備 SOC 電量連續性與充放電限制、以及全系統每小時能量守恆

$\sum P = \sum k + \text{Sell}$ 等等後，最終得到的 constraint_violation_count 為 0。

這項合規表現驗證了本專案所提之「三階段啟發式電力與任務協同調度演算法」的正確性。後半段設計的「實體發電來源動態重組與記帳對齊迴圈」，成功將邏輯層面的任務耗電與物理層面的發電機組實體出力進行了閉合，達能量守恆。

進階動態排程方法 [Only Level 2]

1. 系統效能指標總覽

下表是本虛擬電廠系統在滾動修正、不確定性參數引入等評估數據：

評估維度	核心指標 (Metrics)	實驗量測值	系統效能表現評述
滾動時序與任務效能 (Rolling Timing & Tasks)	硬時限錯失率 (Hard Miss Rate)		在即時環境中鎖定日前排程時段抵抗綠能不穩定參數的波動。 軟/硬時限任務皆為零逾期。
	軟時限錯失率 (Soft Miss Rate)	0.0%	
	平均逾期時間 (Avg Tardiness)	0.0 hr	
	最大相對回應時間 (Max Response Time)	13.0 hr	
	完成時間抖動率 (Jitter)	1.20 hr	
動態強韌度與准入 (Dynamic Robustness)	突發任務總數 (Sporadic Jobs)	2	承接日前准入決策。即使實際 PV 發生 ±10% 預測失真，亦未對既有排程造成時限擾動。
	突發任務接受率 (Acceptance Rate)	100.0%	
	接受後違規率 (Post-Violation Rate)	0.0%	
	突發任務價值率 (Value Rate)	1.0	
經濟與實體損耗優化 (Economics & Battery Realism)	傳統機組總燃料成本 (Generator Cost)		計入 95%/90% 充放電效率損失、電池壽命老化成本與違約金代價。
	電力市場售電收益 (Market Revenue)	\$588,870.0	
	儲能設備老化成本 (Battery Aging Cost)	\$518,078.0	
		\$780.0	
	日前承諾不足處罰 (Shortfall Penalty)	\$247.98	
	最終目標函數值 (Objective Value)	\$71,819.98	

評估維度	核心指標 (Metrics)	實驗量測值	系統效能表現評述
高維度限制式合規 (Advanced Constraints)	任務層級違規數 (Job Violations)	0	通過包含「同時間不可充放電」等約束，驗證滾動修正模型。
	動態物理約束違規數 (Hourly Violations)	0	
	進階總違規計數 (Total Violations)	0	

2. 滾動時序與時限可行性剖析 (Rolling Timing & Scheduling Feasibility)

Level 2在引入 ROLLING_WINDOW = 3 的滾動修正後，表現如下：

2-1. 時限與逾期雙重零錯失 (0% Miss Rate & 0.0 Tardiness)：

硬時限與軟時限任務的錯失率維持在 **0.0%**。由於即時修正並未盲目推翻既有的工作排程組合，日前建立的時間框架可行性得以延續，確保非週期性任務在時限內完成。

2-2. 時序響應與極低運算開銷 (Response Time & Jitter)：

系統整體的平均回應時間為 **4.0 小時**（其中週期性任務之平均回應時間為 **2.97 小時**），最大回應時間控制在 **13.0 小時**。完成時間抖動率（Jitter）由 Level 1 的 1.65 小時收斂至 **1.20 小時**。這是因為 ROLLING_WINDOW = 3 的修正提升系統對時間推進與環境變化的敏感度。

3. 不確定性下之突發任務強韌度與准入效能 (Sporadic Job Robustness & Acceptance)

即時運行環境放寬了再生能源完全準確之假設，系統引入了不確定因子

$\theta_{r,t} = 1 + 0.1\sin(2\pi t / 24)$ ，模擬實際光伏（Actual PV）在 ±10% 內呈正弦變動之波動情境：

3-1. 波動環境下的准入：

日前階段被接納之 2 個突發任務，在即時環境下面臨了實際 PV 出力下降

10% 的極端時段。實驗結果顯示，系統並未因綠能供給失真而發生突發任務被迫中斷、剔除或逾期的現象。

3-2. 零後續違規率：

接受後違規率維持在 **0.0%**。演算法在日前排程中預留之傳統機組與儲能備用容量。當

ROLLING_WINDOW = 3 的即時視窗偵測到再生能源短缺時，傳統機組釋放發電容量，或在合規範圍內動用儲能系統補足電能缺口，展現對不確定性環境的防禦力。

4. 真實儲能損耗與市場承諾套利折衷分析

Level 2 核心優化目標函數值

F_{adv} （objective_value = **\$71,819.98**）納入了更貼近現實世界的實體資產折舊與違約金：

Objective Value = Generator Cost – Market Revenue + Battery Aging Cost + Sell Commitme

4-1. 多目標成本與套利利潤分析：

在此環境下傳統機組發電成本為 **\$588,870.0**，市場售電收益為 **\$518,078.0**。此外，系統累計了 **\$780.0** 的儲能設備老化成本（每 MWh 充放電吞吐量課徵 \$5.0）以及 **\$247.98** 的日前售電承諾不足懲罰金（針對即時售電缺口課徵 25% 的 Cancellation Penalty）。

4-2. 放寬非理想電池物理約束下之市場價格感知行為：

本階段系統放寬了電池理想化假設，納入了充電效率（

$\eta_{ch} = 0.95$ ）、放電效率（ $\eta_{dis} = 0.90$ ）與每小時 0.05% 的自放電損耗。

- 系統僅在電價低於 30% 分位數或存在再生能源賸餘時充電，並藉由 `ROLLING_WINDOW = 3` 的時序動態平滑綠能波動。
- 只有當市場即時電價顯著大於「實體轉換損耗 + c_{age} 老化成本」時，才啟動電池放電售電。
- 透過此機制，系統將日前售電違約金壓制在 **\$247.98**。

5. 限制式合規性結論

當系統放寬假設，約束邊界變得嚴苛。本系統在針對 72 小時全量動態數據的掃描後，`advanced_constraint_violation_count` 為 **0**。

不論是傳統機組的發電升降載率限制（Ramp Limits）、電池 SOC 的電量連續性、SOC 相依之充放電倍率限制、亦或是「**同時時間內不可充放電**（

$Chg_{b,t} \cdot Dis_{b,t} = 0$ ）」的約束，

在 `ROLLING_WINDOW = 3` 的每小時滾動修正中完全合規。全系統每小時能量平衡達成閉合，即時售電量維持非負值。

討論與心得

本次專題的核心挑戰，是要把課堂中學到的即時系統排程理論，實際應用在虛擬電廠的排程問題上，除了要考慮 periodic、sporadic 和 aperiodic jobs 的 release time、deadline 和 execution time，也要同時處理傳統機組、再生能源與儲能設備之間的能源分配問題。因此，在完成這份專題的過程中，我們不只需要理解即時系統的排程概念，也需要思考如何讓排程結果符合各種限制與目標。

在開發過程中，我們使用 AI 作為輔助工具，幫助我們克服對 Python 程式撰寫不夠熟悉，以及一開始不知道該如何從零建立整個專案架構的困難。AI 協助我們理解作業需求、拆解數學限制式，並在程式撰寫和 debug 時提供方向，不過，在真正進行排程設計和 Level 2 assumption 放寬時，我們仍然保留自己的思考，例如判斷應該如何安排 task set、如何設計 acceptance test、要選擇放寬哪些 assumptions，以及不同參數設定可能會對結果造成什麼影響。

在 Level 2 的部分，我們也有請 AI 協助思考可以使用哪些動態排程或調度方法，再根據我們自己的目標和專題需求，選擇較適合的設計，整個過程中，我們覺得 AI 確實是一個很強大的輔助工具，可以幫助我們更快理解問題、整理想法，也能加速程式開發與除錯，但同時我們也發現，如果自己沒有先釐清目標，或沒有能力判斷 AI 給出的內容是否合理，就很容易只是被 AI 的答案帶著走。

使用到的 AI 工具

在本專題中，我們的主要AI工具包括：

OpenAI ChatGPT (GPT-4o)：初期輔助我們理解整份專題要求，例如數學限制式的語意邏輯拆解、在python實做上提供低錯誤的編碼 & Debug。

Anthropic Claude 3.5 Sonnet：專精於複雜演算法架構的模組化設計，提供了高度精確的代碼架構。

Google Gemini 1.5 Pro：免費額度大，雖較為不擅長編寫代碼，但適合作為搜索引擎、回答一些較不深入複雜的內容。

AI 使用方式與協同開發模式

1. 需求分析與語意收斂

一開始面對專題文件中大量的限制式和評分要求時，我們先利用 AI 協助整理自然語言說明與數學公式，並將它們歸納成比較容易理解的筆記。以方便在開始撰寫程式前，先大致了解每個 constraint 的意思，以及它在程式中可能要如何被檢查。

2. 程式原型建立

本專題主要分成三個部分：任務生成 `task_generator.py`、排程 `scheduler.py`，以及評估 `evaluator.py`。我們先請 AI 協助思考每個部分大概需要哪些函式和功能，再根據這些架構逐步完成程式。完成初版後，我們會執行程式並觀察輸出結果，再透過 `evaluator` 檢查是否滿足作業要求和限制式。

3. 程式碼審查、錯誤定位與迭代修正

當程式執行出現錯誤，或 `evaluator` 回報某些 constraint violation 時，我們會把錯誤訊息、相關程式片段，以及輸出的 JSON 結果提供給 AI，請它協助判斷可能是哪個部分出問題。接著，我們再根據 AI 給出的方向進行檢查與修改。這個過程雖然還是需要我們自己確認邏輯是否正確，但確實幫助我們更快找到問題，也讓我們更理解每個限制式在程式中的實際意義。

整體而言，這次專題讓我們學到的不只是如何完成一個排程系統，也讓我們更了解如何和 AI 協同開發。AI 可以幫助我們加速理解、建立架構、撰寫程式和 debug，但最後的設計選擇、參數調整、模型假設和結果解讀，仍然需要由我們自己判斷。